

What is a Jenkins Pipeline?

A **Jenkins Pipeline** is a **suite of automated processes that define how software is built, tested, and deployed** in a Continuous Integration / Continuous Delivery (CI/CD) workflow.

It allows developers to **define the entire build process as code** using a file called **Jenkinsfile**.

💡 In simple terms:

A Jenkins Pipeline describes **all the steps required to build and deliver an application automatically**.

Typical stages in a pipeline include:

- **Build** – Compile the code
- **Test** – Run automated tests
- **Package** – Create deployable artifacts
- **Deploy** – Deploy to staging or production

Example pipeline flow:

Code Commit → Build → Test → Package → Deploy

Benefits of Jenkins Pipeline:

- Pipeline **as Code**
- Easy to **version control**
- Supports **complex workflows**
- Enables **CI/CD automation**
- Better **visualization of build stages**

Different Ways to Write a Jenkins Pipeline

In **Jenkins**, pipelines can be written in **two main ways**: **Declarative Pipeline** and **Scripted Pipeline**. Both methods define the steps required to build, test, and deploy an application automatically.

1. Declarative Pipeline:

Declarative Pipeline uses a simple and structured syntax. It is the most commonly used and recommended approach because it is easy to read, write, and maintain. It uses predefined sections such as **pipeline**, **agent**, **stages**, and **steps** to define the workflow. This format helps developers clearly organize different stages like build, test, and deploy.

2. Scripted Pipeline:

Scripted Pipeline is written using **Groovy** scripting language. It provides more flexibility and control over the pipeline logic. Developers can write complex conditions, loops, and custom logic using Groovy code. However, it is more complex compared to the Declarative Pipeline.

1. Create and build Pipeline Hello World with pipeline script

New Item

Enter an item name

Arthak Exp7

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Hello') {
6       steps {
7         echo 'Hello World'
8       }
9     }
10  }
11 }
12
```

Hello World

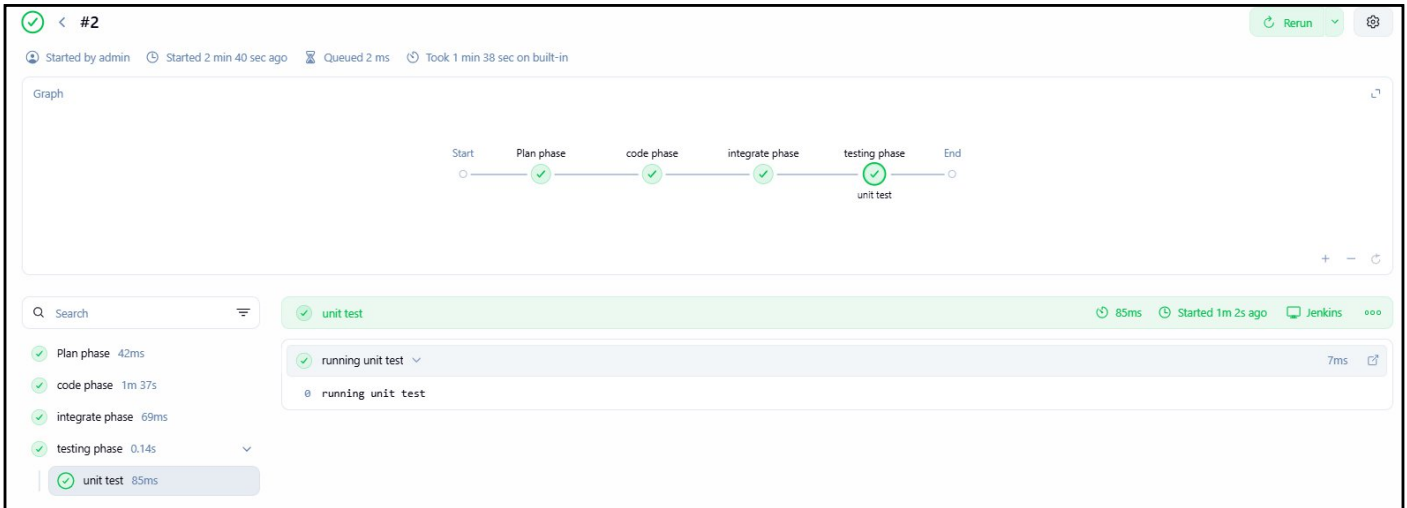
☒ Use Groovy Sandbox ?

Save

Apply

Console

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Tanish Exp7
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Hello)
[Pipeline] echo
Hello World
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Pipeline Steps

stage - (84 ms in block)	Plan phase	▶	✓
stage block (Plan phase) - (42 ms in block)			✓
echo - (7 ms in self)	This is Jenkins Pipeline	▶	✓
stage - (1 min 37 sec in block)	code phase	▶	✓
stage block (code phase) - (1 min 37 sec in block)			✓
input - (1 min 37 sec in self)	Do you want to continue?	▶	✓
stage - (0.1 sec in block)	integrate phase	▶	✓
stage block (integrate phase) - (69 ms in block)			✓
echo - (7 ms in self)	Integration test passed	▶	✓
stage - (0.17 sec in block)	testing phase	▶	✓
stage block (testing phase) - (0.14 sec in block)			✓
parallel - (0.11 sec in block)		▶	✓
parallel block (Branch: unit test) - (85 ms in block)			✓
stage - (59 ms in block)	unit test	▶	✓
stage block (unit test) - (31 ms in block)			✓
echo - (7 ms in self)	running unit test	▶	✓

3. Pipeline Project with GitHub.

jenkins-pipeline-tutorial Public Watch 0

master 1 Branch 0 Tags Go to file Add file Code

chrismld Run only crucial integration tests d6d8a4b · 7 years ago 7 Commits

- hello-world Run only crucial integration tests 7 years ago
- README.md Initial commit 7 years ago

README

jenkins-pipeline-tutorial

Jenkins Pipeline Tutorial

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/ArthakSawant-06/jenkins-pipeline-tutorial.git

! Please enter Git repository.

Credentials

- none -

+ Add

Advanced

Script Path ?

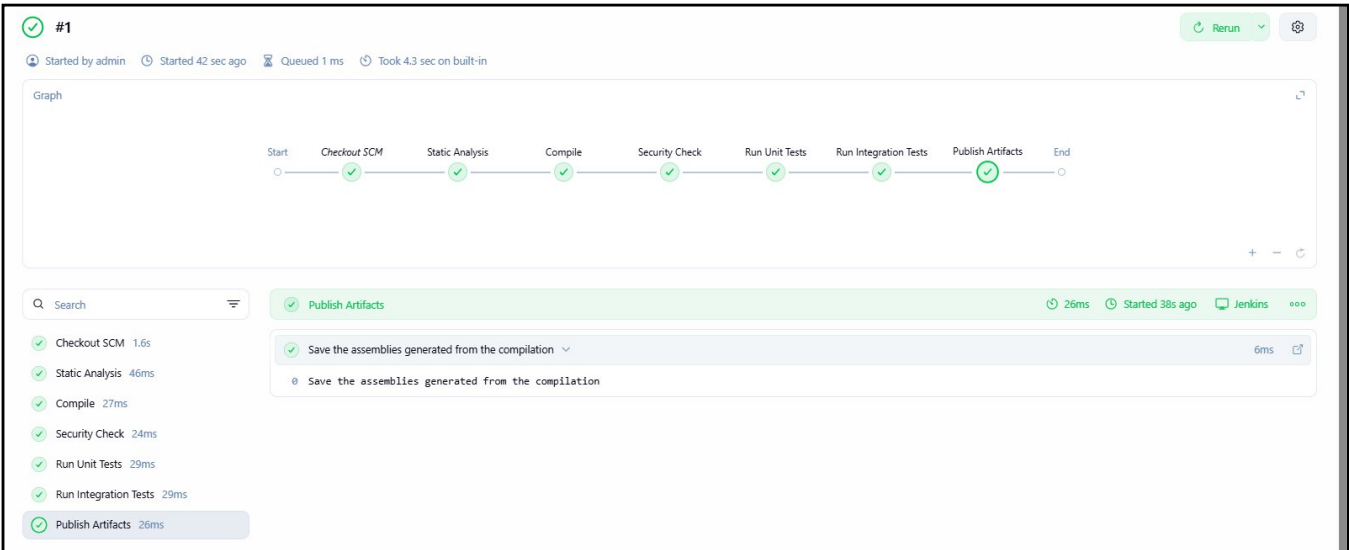
hello-world/Jenkinsfile

ERROR: Failed to load help file: Server Error

☒ Lightweight checkout ?

[Pipeline Syntax](#)

```
Run only crucial integration tests from the source code
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Publish Artifacts)
[Pipeline] echo
Save the assemblies generated from the compilation
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



stage - (61 ms in block)	Security Check		
stage block (Security Check) - (24 ms in block)			
echo - (6 ms in self)	Run the security check against the application		
stage - (67 ms in block)	Run Unit Tests		
stage block (Run Unit Tests) - (29 ms in block)			
echo - (8 ms in self)	Run unit tests from the source code		
stage - (74 ms in block)	Run Integration Tests		
stage block (Run Integration Tests) - (29 ms in block)			
echo - (6 ms in self)	Run only crucial integration tests from the source code		
stage - (49 ms in block)	Publish Artifacts		
stage block (Publish Artifacts) - (26 ms in block)			
echo - (6 ms in self)	Save the assemblies generated from the compilation		

4. Create pipeline project with pipeline script from SCM

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/ArthakSawant-06/EXP7.git

! Please enter Git repository.

Credentials

- none -

+ Add

Advanced

***/main is imp here**

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

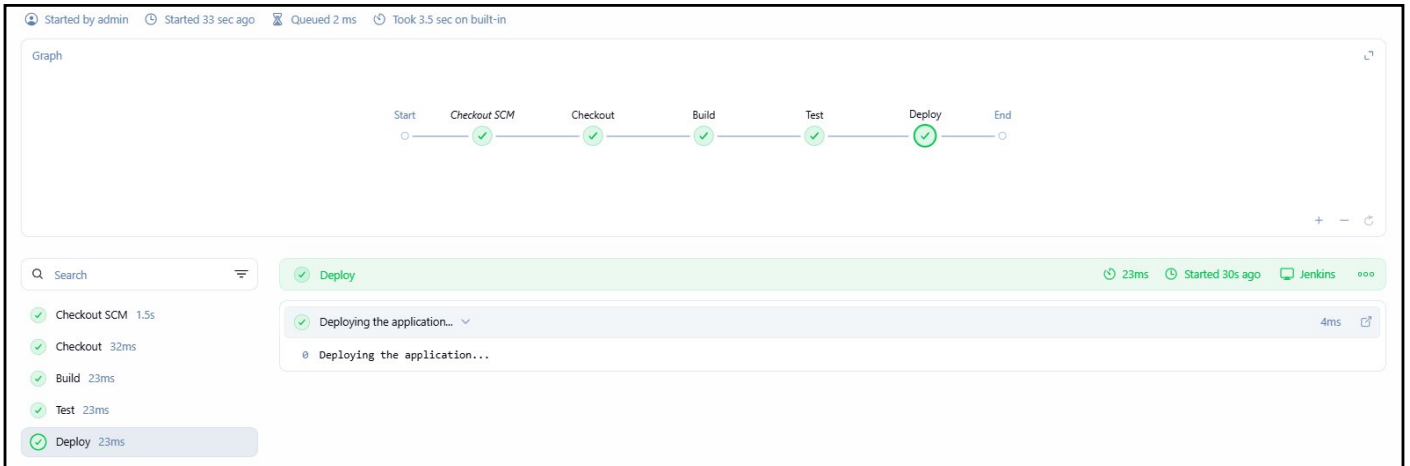
☒ Lightweight checkout ?

[Pipeline Syntax](#)

```

Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```



stage block (Checkout) - (32 ms in block)		✓
echo - (6 ms in self)	Checking out code from repository	✓
stage - (55 ms in block)	Build	✓
stage block (Build) - (23 ms in block)		✓
echo - (5 ms in self)	Building the application...	✓
stage - (56 ms in block)	Test	✓
stage block (Test) - (23 ms in block)		✓
echo - (5 ms in self)	Running tests...	✓
stage - (47 ms in block)	Deploy	✓
stage block (Deploy) - (23 ms in block)		✓
echo - (4 ms in self)	Deploying the application...	✓

Pipeline Script

```

pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo 'Building application...'
      }
    }
  }
}

```

```

stage('Test') {
  steps {
    echo 'Running tests...'
  }
}

stage('Deploy') {
  steps {
    echo 'Deploying application...'
  }
}
}

```

--	--